System Architecture

Customer Insights Segment Sankey

Draft segment preview and member analysis with Dynamics 365, Dataverse, Azure, and Microsoft Fabric

Document	Value
Version	1.3
Date	August 21, 2026
Status	As-built architecture with operational and security notes
Audience	Architecture, development, operations, security, and product owners
Primary format	Microsoft Word; diagrams additionally editable in Excalidraw

Note: Secrets and API keys are deliberately not part of this document.

1. Management Summary

The solution extends the draft segment editor of Dynamics 365 Customer Insights – Journeys with an interactive Sankey preview. For each cumulative filter stage it calculates the member count, visualizes inflows and outflows, and enables drill-down into the contacts still contained, removed, or added.

The actual set evaluation takes place entirely in Microsoft Fabric. The Dataverse plugin reads and validates the draft MQL, builds a typed abstract syntax tree, and hands it to a secured ASP.NET Core API. This API compiles profile, relationship, consent, behavioral, and set operations into parameterized Fabric SQL CTEs.

For the overview, only aggregated COUNT_BIG values leave Fabric. For member drill-down, Fabric returns at most 50 contact IDs per page; Dataverse then adds the visible fields in the context of the signed-in user. This preserves record- and field-level security.

1.1 Key Architecture Decisions

- Fabric is the sole engine for segment sets and set operations.
- Dataverse remains the system of record for draft MQL, metadata, and security-checked contact data.
- Journeys events and required Dataverse tables are provisioned in a shared Serving Lakehouse.
- A tenant-owned browser Setup Center provisions Azure and Fabric through delegated OAuth with PKCE.
- The browser UI never receives an API key or large ID sets.
- Short-lived, encrypted tokens bind the member query to segment, stage, filter, sorting, and continuation.
- Errors are reported explicitly; a count of 0 is only shown after a successful evaluation.

1.2 Current Functional Scope

Area	Implementation
Segment logic	PROFILE, relationships, consent, interaction, nested/static segments
Set operations	INTERSECT, UNION, and EXCEPT
Visualization	Cumulative Sankey stages, removed and added bubbles, result
Members	Search, field filters, sorting, 50-item keyset paging, contact navigation
Security	Dataverse security trimming, field-level security, server-side API key, managed identity
Scaling	Counts independent of profile count; no 250,000 ID limit

2. System Context

The segment designer operates exclusively within the Customer Insights model-driven app. A command opens a Dynamics side pane. The pane calls global Dataverse Custom APIs, which coordinate Dataverse and the Azure/Fabric evaluation.

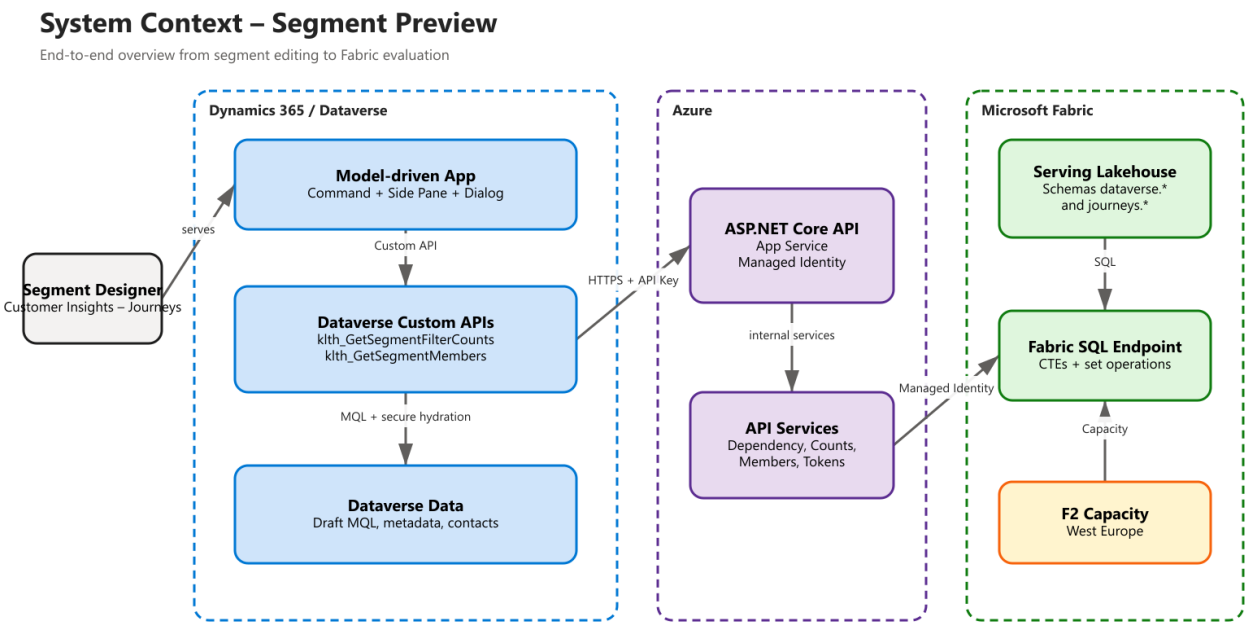

Figure 1: System context and key platform boundaries

2.1 Participating Platforms and Responsibilities

Platform	Responsibility
Dynamics 365 Customer Insights – Journeys	Segment editor, draft MQL, user interaction
Dataverse	Segment definition, metadata, environment variables, contacts, and security model
Azure App Service	Secured segment engine and integration endpoints
Azure Storage and networking	Private immutable API package, VNet integration, private endpoint, and private DNS
Azure Container Instance	One-time managed-identity package download and SHA-256 verification
Microsoft Fabric	Serving Lakehouse, SQL evaluation, set operations, and event data
Azure Fabric Capacity	Compute capacity for Lakehouse, REST, and SQL endpoint

2.2 User Interactions

1. The user opens or edits a saved draft segment definition.
2. The command opens the "Segment preview" side pane and passes the segment ID and name.
3. The pane saves changed form data before evaluation.
4. The count API returns ordered stage counts and a short-lived evaluation token.
5. Clicking a stage, a red/green bubble, or the result opens the members dialog.
6. Clicking a contact name opens the Dataverse contact record.

2.3 High-Level Deployment and Infrastructure Mapping

The solution consists of four clearly separated runtime areas. The HTML/JavaScript web resources are stored in Dataverse but run in the user's browser. The C# plugins are classic server-side Dataverse IPlugin implementations and run synchronously in the plugin runtime provided by Dataverse. They are not Azure Functions.

In Azure, a standalone ASP.NET Core .NET 8 REST API runs as a Linux Web App on Azure App Service. It provides the endpoints for dependencies, counts, members, and health. There is no Azure Function and no Function App host in this architecture.

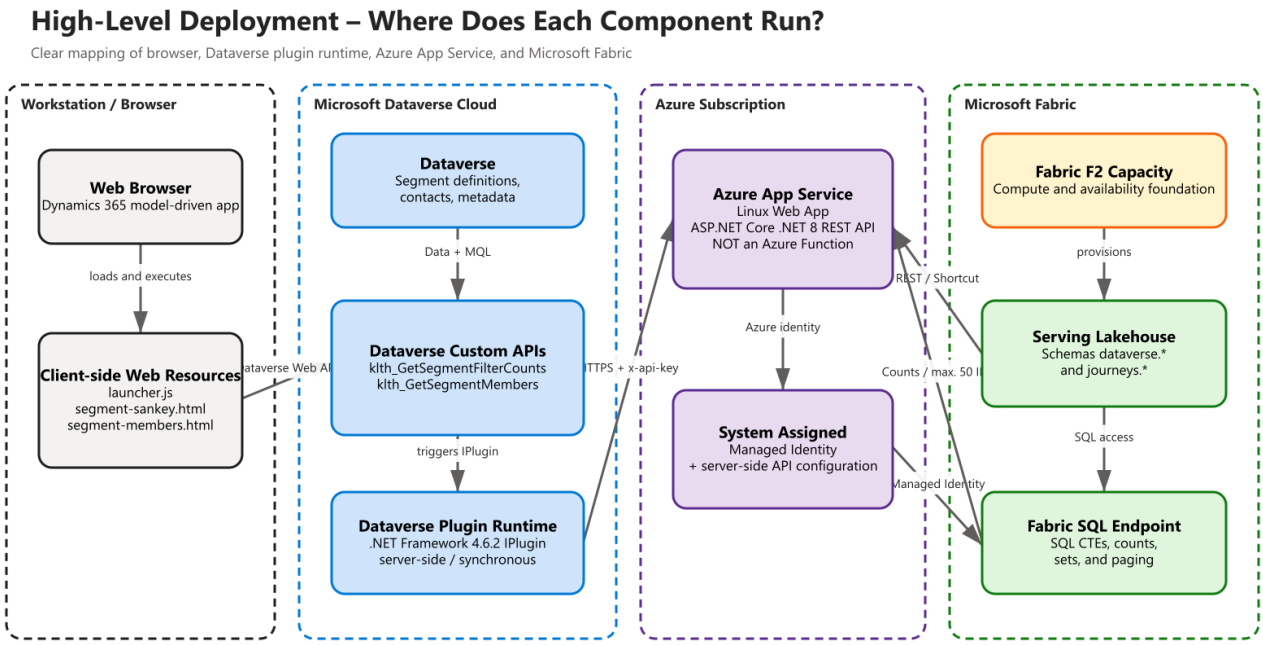

Figure 2: Physical runtime mapping of the components

Component	Technology	Runtime / Infrastructure	Key Clarification
Sankey and members dialog	HTML, CSS, JavaScript	Browser; files stored as Dataverse web resources	Client-side UI, no dedicated web server
Count and members plugin	C# IPlugin, .NET Framework 4.6.2	Server-side Dataverse plugin runtime	No browser code and no Azure Function
Segment engine API	ASP.NET Core, .NET 8	Azure App Service Linux Web App	Regular REST web API, explicitly not a Function App
Set computation	Fabric SQL	Fabric SQL Endpoint on F2 Capacity	Database-native CTE and set operations
Serving data	Delta/OneLake shortcuts	Fabric Serving Lakehouse	dataverse.* and journeys.*

2.4 Request Flow: Custom API, Plugin, Azure, and Fabric

In this solution, the Dataverse Custom API is the registered interface through which the web resource specifically triggers the associated plugin. The actual business and integration logic resides in the C# plugin. This plugin reads Dataverse data and then calls the REST API on the Azure Web App.

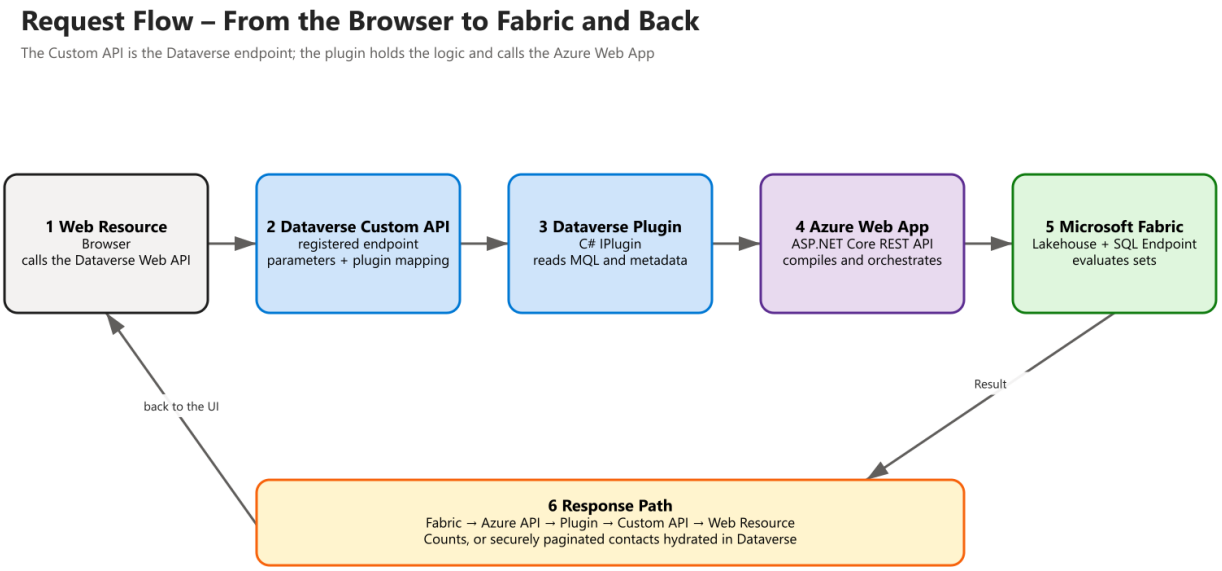

Figure 3: Outbound and return path of a segment request

7. The JavaScript web resource calls the Dataverse Custom API via the authenticated Dataverse Web API.
8. Dataverse validates the registered endpoint and its input/output parameters.
9. The Custom API triggers the associated C# IPlugin type in the Dataverse plugin runtime.
10. The plugin reads draft MQL, metadata, segment references, and server-side environment variables.
11. The plugin calls the ASP.NET Core API on Azure App Service over HTTPS with a server-side x-api-key.
12. The Azure API authenticates to Microsoft Fabric via managed identity and performs shortcut or SQL operations.
13. Fabric returns counts or at most 50 profile IDs to the Azure API.
14. The plugin hydrates member IDs as the currently signed-in Dataverse user and returns the response through the Custom API and web resource.

2.5 Browser-Only Provisioning

After importing the Dataverse solution, the administrator opens the Segment Preview Setup web resource. A customer-owned, single-tenant Entra SPA registration performs authorization-code flow with PKCE. Delegated Azure Resource Manager and Fabric tokens remain in browser memory and are audience-specific; no client secret is stored in Dataverse.

15. The Setup Center validates the target tenant, subscription, resource group, Fabric workspace, Lakehouses, SQL endpoint, and cloud connection.

16. The embedded Fabric notebook definition is parameterized, created or updated through Fabric REST, and scheduled idempotently.
17. ARM deploys App Service, private package storage, VNet integration, private Blob endpoint and DNS, managed identities, and a one-time Azure Container Instance. Setup waits for the copy command's terminal exit code, retries one failure, and proceeds only after exit code 0. Resume state records the package release identifier, so importing a newer Solution reruns the Azure package deployment even when the API version and digest are unchanged.
18. The copy container downloads the pinned API release, verifies its SHA-256, and uploads the immutable digest-named blob with Entra authentication. Shared keys and SAS tokens are disabled.
19. The Web App reads the private blob through its system-assigned identity. The Setup Center waits for the restricted health endpoint before writing the API URL and generated key to Dataverse.
20. The Web App identity receives Fabric Contributor, missing Dataverse shortcuts are provisioned, and the final setup status must report Ready.

Provisioning asset	Ownership	Security property
SPA registration	Customer tenant	Single-tenant, public SPA, PKCE, no client secret
API package blob	Customer subscription	Private endpoint, managed-identity read, immutable digest name
Copy container	Customer subscription	One-time execution, scoped user-assigned identity
Web App	Customer subscription	HTTPS-only, VNet-integrated, system-assigned identity
Notebook and schedule	Customer Fabric workspace	Embedded definition, idempotent REST publication

3. Logical Architecture

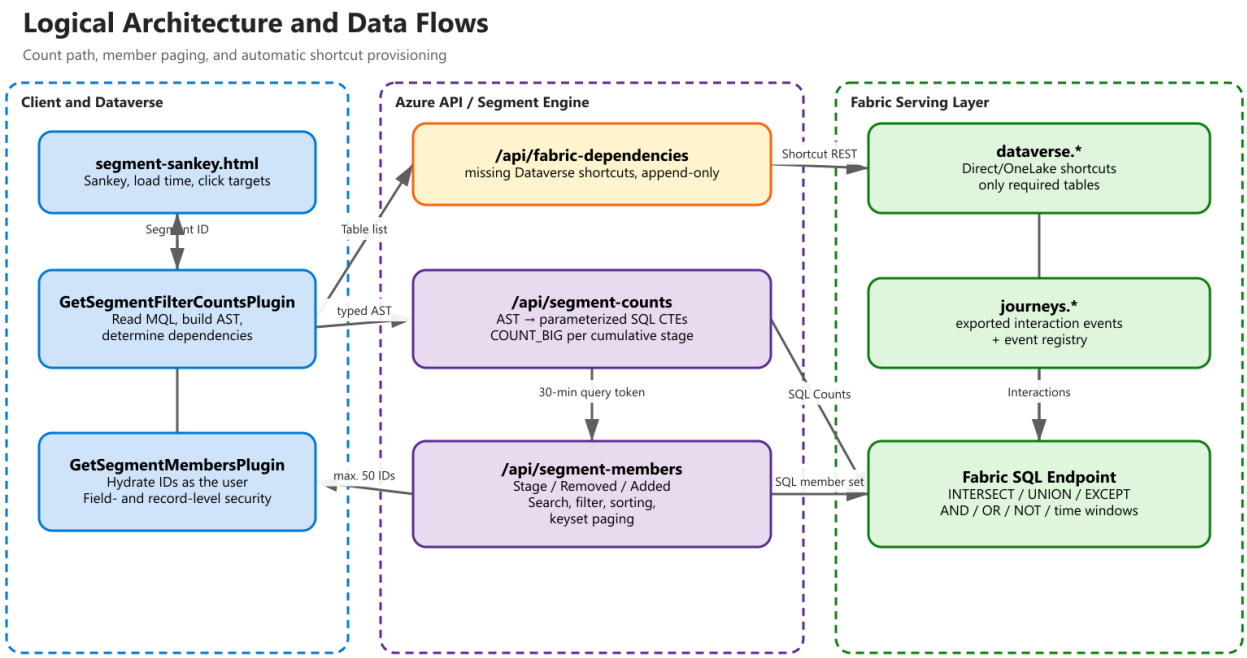

Figure 4: Count path, member paging, and shortcut provisioning

3.1 Client Components

Component	Task
cis_SegmentSankeyLauncher.js	Opens and navigates the side pane; synchronizes the active segment definition
segment-sankey.html	Visualizes counts, retention, inflows/outflows, and end-to-end load time
segment-members.html	Dialog for search, filter, sorting, paging, and record navigation

3.2 Dataverse Components

Component	Task
klth_GetSegmentFilterCounts	Global Custom API for the full draft evaluation
GetSegmentFilterCountsPlugin	Read and validate MQL, build dependencies and AST
klth_GetSegmentMembers	Global Custom API for a single members page
GetSegmentMembersPlugin	Hydrate Fabric IDs from Dataverse as the calling user
Environment Variables	Server-side API base URL and API key

3.3 Azure API Components

Endpoint	Responsibility
/api/health	Liveness of the ASP.NET Core application
Unified /api/segment-counts	Optimistic count; repair only confirmed missing tables

Endpoint	Responsibility
/api/segment-counts	Compile the typed AST into SQL CTEs and execute stage counts
/api/segment-members	Execute the Stage/Removed/Added set with search, filter, sorting, and paging

3.4 Fabric Components

Component	Task
Serving Lakehouse	Shared, SQL-queryable serving layer
Schema dataverse.*	Direct/OneLake shortcuts to required Dataverse mirror tables
Schema journeys.*	Customer Insights Journeys events
SQL Endpoint	Execution of parameterized CTEs and set operations
Bootstrap Notebook	Daily synchronization of new event and Dataverse shortcuts
Registries	journeys._event_registry and dataverse._shortcut_registry

4. Data Architecture

4.1 Data Sources

The solution unifies two functional data sources in a Serving Lakehouse. Dataverse provides profiles, relationships, and consent tables. Customer Insights – Journeys exports interaction events into Delta structures, which become visible as tables in the journeys schema.

Source	Serving Schema	Content	Refresh
Dataverse Link to Fabric / Mirror	dataverse.*	Contact, relationships, consent, and other required tables	Direct/OneLake shortcuts; on-demand
Customer Insights Journeys Export	journeys.*	Email, link, event, and other interactions	Export + daily bootstrap notebook

4.2 Automatic Dependency Resolution

The MQL dependency resolver analyzes profile filters, RELATE relationships, consent, interactions, nested segments, and static segment references once while building the typed count request. The query and required Dataverse table list are sent together to /api/segment-counts.

- A successful SQL evaluation returns immediately. Only a confirmed missing-table failure triggers targeted shortcut reconciliation, SQL metadata refresh, and one retry.
- Source and Serving shortcut listings run concurrently and stop when all requested tables are found. Exact targets are preserved, stale targets are repaired, and successful validation runs in a deduplicated background queue. At startup, capacity readiness, the shared Fabric token, and SQL connectivity warm concurrently; confirmed active capacity state is cached briefly.
- A limit of 20 tables applies per call.

- Table and column names are validated via INFORMATION_SCHEMA.

4.3 SQL Compilation

The SQL compiler generates named CTEs for base profile, filter groups, relationships, interactions, and set operations. Identifiers are quoted from the validated catalog; all business values are parameterized.

- Supported logic: AND, OR, NOT, comparison operators, IN, CONTAINS, and Count().
- Time windows: UTCMINUTES, UTCHOURS, UTCDAYS, UTCWEEKS, UTCMONTHS, and UTCYEARS.
- Text semantics: CONTAINS is case-insensitive but accent-sensitive.
- Counts use COUNT_BIG(*); no download of full profile sets.

4.4 Stage Semantics

Mode	Set Meaning
stage	Cumulative members of the selected stage
removed	Previous stage EXCEPT selected stage
added	Selected stage EXCEPT previous stage
result	Final cumulative stage set

5. Count Evaluation Flow

21. The side pane calls klth_GetSegmentFilterCounts with the segment definition ID.
22. The plugin reads the current draft MQL and resolves referenced segments and relationship metadata.
23. The dependency resolver determines required Dataverse tables and Journeys events.
24. The Azure API starts capacity readiness and SQL evaluation concurrently, and provisions only confirmed missing Dataverse shortcuts.
25. The plugin sends the validated typed AST and required table list in the same /api/segment-counts request.
26. The catalog service resolves allowed tables and columns via INFORMATION_SCHEMA.
27. The compiler generates parameterized SQL CTEs for all cumulative stages.
28. Fabric executes the query and returns only COUNT_BIG per stage.
29. The API also issues an encrypted query token valid for 30 minutes.
30. The plugin returns ordered stages, the evaluation token, and a timestamp to the pane.

The pane draws Sankey flows and calculates losses, additions, and retention. It renders accessible summary and flow placeholders immediately while counts are pending; an in-place refresh retains the last successful visualization until the replacement result is complete.

5.1 Example of a Verified Result

Stage	Semantics	Count
0	Active contacts	640
1	Has email address	625
2	Profile intersection	407
3	Behavioral intersection	2
4	Union with static segment	75

6. Members View Flow

31. The count result is available in the pane with an evaluation token and explicit stage order.
32. The browser stores the dialog context under a random key in window.top; the token never appears in the URL.
33. The dialog calls klth_GetSegmentMembers with stage, mode, search, filter, sorting, and an optional continuation token.
34. The plugin translates evaluationToken into queryToken and calls /api/segment-members server-side.
35. Fabric verifies the token and context binding, computes the requested set, and reads TOP(pageSize + 1).
36. Fabric returns at most 50 profile IDs plus a new continuation token.
37. The Dataverse plugin reads the contacts using CreateOrganizationService(context.UserId).
38. Inaccessible records and secured fields are removed by Dataverse.
39. The browser receives only the visible fields and can open the contact record.

6.1 Paging Approach

- Maximum of 50 IDs or visible contacts per page.
- Keyset paging instead of OFFSET for stable runtime with large data volumes.
- Sort key: null rank, sort value, and profile ID.
- The continuation token binds the query hash, stage, mode, filter, sorting, and last key.

6.2 Supported Fields

Field	Search	Filter	Sort	Display
fullname	Yes	Yes	Yes	Yes
firstname	Yes	Yes	Yes	Yes
lastname	Yes	Yes	Yes	Yes
emailaddress1	Yes	Yes	Yes	Yes
telephone1	Yes	Yes	Yes	Yes
address1_city	Yes	Yes	Yes	Yes

Field	Search	Filter	Sort	Display
companyname	No	No	No	Yes; after Dataverse hydration

7. Security Architecture

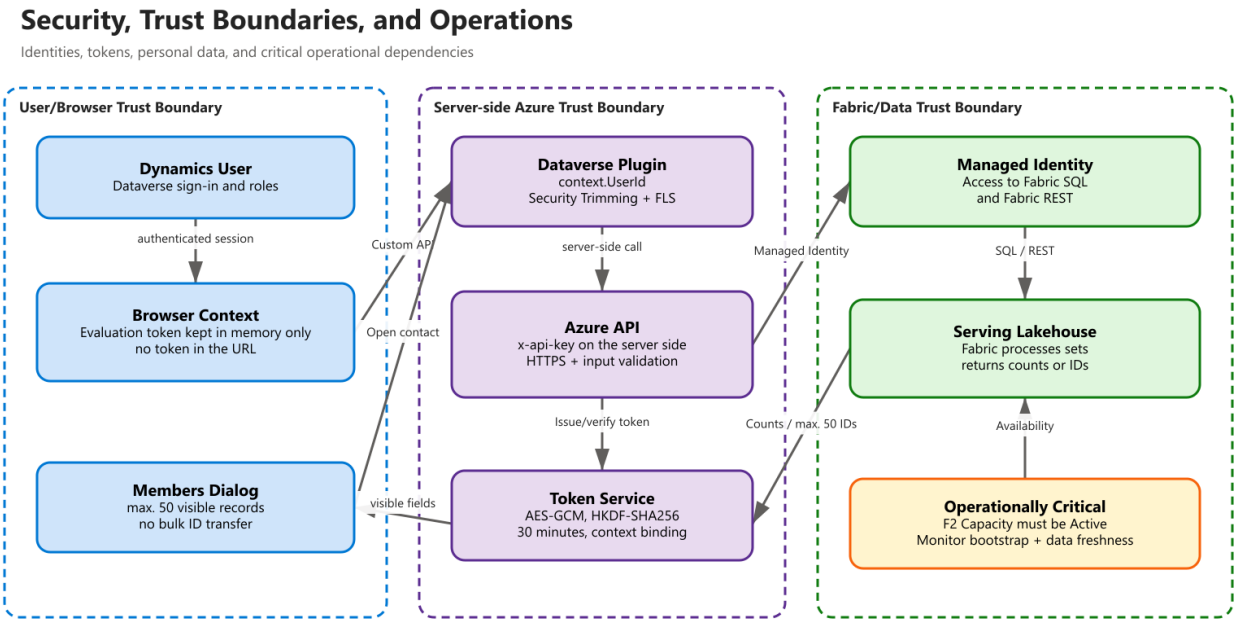

Figure 5: Trust boundaries, identities, tokens, and operational dependencies

7.1 Identities and Authentication

Connection	Authentication
Browser → Dataverse	Current Dynamics 365 user session
Dataverse Plugin → Azure API	Server-side x-api-key from environment variable
Azure API → Fabric SQL / REST	System Assigned Managed Identity
Plugin → Dataverse contacts	context.UserId of the calling user

7.2 Data Protection and Permissions

- The count path never transmits contact IDs or personal fields.
- The Fabric members response contains only profile IDs.
- Name, email, phone, city, and company are added only in Dataverse.
- Dataverse record-level security and field-level security remain active.
- The browser receives at most 50 visible records per page.
- The API key is never exposed to JavaScript or in URLs.

7.3 Token Security

Property	Implementation
Encryption	AES-GCM with a random 12-byte nonce
Key derivation	HKDF-SHA256 from the server-side API key
Key separation	Separate derived keys for query and continuation tokens
Validity	30 minutes
Tamper protection	Authenticated encryption and canonical Base64URL verification
Context binding	Query hash, stage, mode, search/filter, sorting, and last key

7.4 Security Risks and Mitigations

Risk	Mitigation
SQL injection	Catalog allow-listing, identifier quoting, and parameterized values
Token tampering	AES-GCM, context binding, expiry, and canonical encoding
Data exfiltration	Counts-only on the main path; max. 50 IDs; Dataverse hydration
Permission bypass	Query as context.UserId; no server-side full-access hydration
Secret leakage	API key only in Dataverse environment variable and app configuration

8. Physical Deployment

8.1 Dataverse

Property	Value
Organization	orga7223214.crm4.dynamics.com
Organization ID	8b3bfb3a-7c11-f111-afc0-000d3ab2a775
Plugin Assembly	CustomerInsightsSegmentSankey, Version 0.0.15.0
Plugin Assembly ID	458b9dc7-1d9a-f111-b8dc-7ced8d762587
Custom APIs	klth_GetSegmentFilterCounts; klth_GetSegmentMembers
Publisher Prefix	klth

8.2 Azure

Property	Value
Resource Group	D365DemoTSCE21021953CXCS
Web App	klth-segment-sankey-8b3bfb3a
Runtime	.NET 8 on Linux

Property	Value
Identity	System Assigned Managed Identity
Current Plan	F1 Free, 1 worker
Always On	Currently disabled; not available on F1
TLS	TLS 1.2 minimum
API Base	https://klth-segment-sankey-8b3bfb3a.azurewebsites.net/api/

Operational recommendation: For production-like operation, use at least B1 and enable Always On. The currently observed as-built state is F1 with Always On disabled, which deviates from the recommended configuration.

8.3 Microsoft Fabric

Property	Value
Workspace ID	904ef357-46bc-48b6-ac20-4d26032b0e32
Serving Lakehouse ID	7b68f4e1-2300-4cf9-bf15-2d886efa95e4
SQL Endpoint ID	7b996864-cda7-4817-a75f-dd80d52d5777
Capacity	d365demotsce21021953cxcs, F2, West Europe
Capacity ID	c42ec01a-0d0d-48d4-961c-c93dfbe3312e
Current Status	Active as of August 20, 2026

9. Operations and Monitoring

9.1 Availability Dependencies

Dependency	Impact on Failure	Check
Azure Web App	No count or member queries	/api/health
Fabric Capacity	Lakehouse, shortcuts, and SQL unreachable	Capacity state must be Active
Fabric SQL Endpoint	No segment evaluation	Test query / count endpoint
Dataverse Custom APIs	Pane cannot call the backend	Web API smoke test
Bootstrap Notebook	New events/tables missing	Notebook run + registries
Journeys Export	Behavioral data stale	Data freshness of the journeys tables

9.2 Typical Incident: Capacity Inactive

On August 20, 2026, the message "The existing Fabric shortcuts could not be read" occurred. The Fabric REST API returned EntityNotFound with the detail that capacity c42ec01a-0d0d-48d4-961c-

c93dfbe3312e was unavailable. The capacity state was Inactive. After resuming the F2 capacity, the dependency check, SQL evaluation, and browser preview worked again.

40. Check the capacity state in Fabric or via the Fabric REST API.
41. If Inactive: resume the existing Azure Fabric Capacity.
42. Test /api/fabric-dependencies with contact.
43. Test klth_GetSegmentFilterCounts with a known segment.
44. Refresh the pane and verify counts and load time.

9.3 Logging and Error Principles

- Plugin tracing logs endpoint calls and business errors.
- The Azure API returns machine-readable error codes, user-facing text, and technical details.
- Unsupported fields/events are reported explicitly.
- No silent fallback to sample values in Dataverse.
- A transient SQL timeout -2 is retried once during the initial catalog build.

9.4 Performance

Measurement Point	Observed Range
Web App warm	0.15–0.49 s
Dependency check	0.72–1.94 s
First Fabric SQL call	approx. 5.55 s
Warm Fabric SQL calls	0.31–0.41 s
Full Sankey evaluation	3.74–7.15 s; depends on Fabric warm state

The largest remaining latency comes from Fabric SQL cold starts and metadata/catalog work. A larger App Service plan with Always On only prevents Web App cold starts; Fabric Capacity has a separate availability and warm-start characteristic.

10. Scaling and Limits

10.1 Scaling Characteristics

- Counts transmit only aggregated values, independent of profile count.
- Set operations run close to the database in Fabric instead of in the Dataverse plugin.
- Keyset paging avoids OFFSET costs on deep pages.
- Catalog and connection pooling reduce warm follow-up costs.
- Automatic shortcuts avoid manual table maintenance per segment.

10.2 Known Limits

Limit	Assessment / Mitigation
Very large static segments	IDs are currently passed as a JSON parameter; use a persisted Fabric membership table in the future
Data freshness	Depends on Journeys export, Dataverse mirror, and bootstrap
Capacity paused/inactive	Entire Fabric evaluation unavailable; monitoring required
Company sorting	Not supported, since companyname only comes from Dataverse after Fabric paging
App Service F1	No Always On; switch to B1 or higher for production operation
Editor-internal preview endpoint	Deliberately not used, since it is unsupported and unsuitable for partial filters

10.3 Target State for High Load

- Size Fabric Capacity according to data volume and concurrency.
- Keep the catalog and SQL connections warm; Azure App Service at least B1.
- Materialize static memberships above a defined size in a Fabric table.
- Monitor capacity status, SQL latency, bootstrap success, and data freshness.
- Perform load tests with realistic segments, interactions, and concurrent dialogs.

11. Quality Assurance and Verified State

Check	Result
Fabric unit tests	53/53 passed
Plugin build	Release build succeeded, 0 warnings and 0 errors
Strong name	Token 7723c9b9c78b183b, consistent with the registered assembly
Live counts	640 → 625 → 407 → 2 → 75
Members page	50 records, continuation token, and page 2 without overlap
Search/filter	Live, expected single match in each case
Sorting	Verified live via email
Dialogs	Stage, Removed, Added, and Result opened live
Contact navigation	entityName contact, specific ID, openInNewWindow true
Capacity recovery	Correct counts and browser display again after resume

11.1 Acceptance Criteria

- All stages are ordered and match the full cumulative segment logic.
- Added/removed sets correspond to the directed EXCEPT operations.
- The browser shows no PII from Fabric and no tokens in URLs.

- Dataverse permissions determine the actually visible member rows.
- Error states show an understandable message instead of an apparent zero value.

12. Deployment and Change Process

12.1 Azure API

45. Build the FabricApi sources and run the complete Fabric API test suite.
46. Publish the API and check /api/health.
47. Run direct tests for /api/fabric-dependencies, /api/segment-counts, and /api/segment-members.
48. Verify managed identity permissions and Fabric Capacity.

12.2 Dataverse Plugin and Custom APIs

49. Increment the assembly version and build with the existing strong-name key.
50. Update the assembly content; the fully qualified name must remain unchanged.
51. Register new plugin types and Custom API parameters.
52. Publish customizations.
53. Test the count and members Custom APIs via the Dataverse Web API.

12.3 Web Resources

54. Update segment-sankey.html, segment-members.html, and the launcher.
55. Publish web resources and reload the Dynamics page to switch the cache version.
56. Test the command, pane, all click targets, filters, paging, and contact navigation.

12.4 Rollback

- Re-upload the previous plugin assembly with the identical strong name.
- Restore and publish the previous web resource contents.
- Roll the Azure API back to the previous deployment.
- Do not delete existing Fabric shortcuts; preserve exact targets and repair only stale targets.

13. API Contracts

13.1 Dataverse Count API

```
POST /api/data/v9.2/klth_GetSegmentFilterCounts
{ "klth_segmentid": "<Guid>" }

Response property: klth_resultjson (String)
Content: generatedAt, stages[{order,label,detail,count}], evaluationToken
```

13.2 Dataverse Members API

```
POST /api/data/v9.2/klth_GetSegmentMembers
{ "klth_requestjson": "{...}" }

Request: evaluationToken, stageOrder, viewMode, search, filter,
```

```
        sortField, sortDirection, pageSize, continuationToken
Response: rows[], nextToken, hasMore, generatedAt
```

13.3 Azure API Protection

- All business endpoints expect x-api-key.
- The API base must be an absolute HTTPS URL.
- Payloads are validated for size, tables, fields, operators, and stage context.
- Managed identity authenticates the API to Fabric.

14. Source Code and Artifact Overview

Path	Purpose
CustomApi/GetSegmentFilterCountsPlugin.cs	Dataverse entry point for count evaluation
CustomApi/GetSegmentMembersPlugin.cs	Dataverse security boundary for the members view
CustomApi/FabricSegmentCountClient.cs	AST construction and Fabric count call
CustomApi/FabricDependencyProvisioningClient.cs	Compatibility client for explicit table provisioning
FabricApi/SegmentCountSqlCompiler.cs.txt	CTE and set compiler
FabricApi/FabricSegmentCountService.cs.txt	Count orchestration
FabricApi/FabricSegmentMemberService.cs.txt	Member paging
FabricApi/QueryTokenService.cs.txt	Encrypted query/continuation tokens
webresources/segment-sankey.html	Sankey side pane
webresources/segment-members.html	Members dialog
Fabric/bootstrap-events.py	Serving Lakehouse bootstrap
FabricApi.Tests/	Compiler, token, and serialization tests

14.1 Included Diagram Sources

- documentation/diagrams/01-system-context.excalidraw
- documentation/diagrams/02-logical-architecture.excalidraw
- documentation/diagrams/03-security-operations.excalidraw
- documentation/diagrams/04-high-level-deployment.excalidraw
- documentation/diagrams/05-request-flow.excalidraw

The Excalidraw files are the editable sources. PNG and SVG versions are stored alongside them and can be used for presentations, wikis, and technical reviews.

15. Glossary

Term	Definition
AST	Abstract syntax tree of the validated segment logic
CTE	Common Table Expression; a named SQL subexpression
Draft MQL	Not-yet-published segment definition from the Journeys editor
FLS	Field-Level Security in Dataverse
Keyset Paging	Continuation via last sort value and primary key instead of OFFSET
Managed Identity	Identity managed by Azure with no secret in source code
OneLake Shortcut	Virtual reference to data without a physical copy
Serving Lakehouse	Shared, SQL-queryable layer for profiles and Journeys events
Stage	Cumulative result set after a segment filter or set operation

16. Open Operational Recommendations

57. Scale the App Service back up to at least B1 and enable Always On.
58. Set up an alert for a Fabric Capacity state other than Active.
59. Monitor bootstrap failures and data freshness of the journeys/dataverse registries.
60. Map very large static segments to a persisted Fabric membership table.
61. Adopt technical runbooks for capacity resume, API rollback, and shortcut errors into operations.
62. Update the document whenever APIs, security boundaries, Fabric schemas, or deployment IDs change.